



Agentic AI Bootcamp



Learn to Build AI That Thinks, Plans, and Acts.



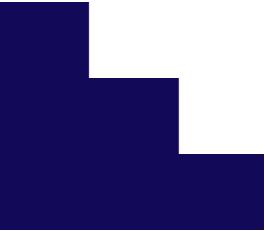
Introduction to Multi-Agent Systems

Prepared by WeCloudData





Agenda



01

Why multi-agent systems exist

02

Limitations of Single Agents

03

Role Specialization

04

Task Delegation Patterns

05

Basic Communication Patterns

Why Multi-Agent Systems Exist



Why Multi-Agent Systems Exist

AI tasks are growing faster than single models can handle them

- The first generation of LLM applications handled narrow, one-shot tasks like ‘Summarize this paragraph.’, ‘Answer this question.’ These fit cleanly inside a single prompt and a single response.
- The second generation, the one being built right now handles entire **workflows**. For example research a market, write a competitive analysis, verify the facts, format the report, and send it. These workflows span thousands of tokens, multiple tools, and multiple cognitive modes (analytical, creative, critical).
- Asking one agent to handle all of this is like asking one employee to be your lawyer, accountant, engineer, copywriter, and quality reviewer simultaneously, in a single conversation, without forgetting anything, without making mistakes, and without getting tired.



Why Multi-Agent Systems Exist

What exactly is a Multi-Agent System?

A **multi-agent system (MAS)** is an architecture in which two or more AI agents, each with its own identity, tools, memory, and responsibilities, work together to complete tasks that exceed what any single agent could accomplish alone.

EACH AGENT HAS:

- A defined role or persona (via system prompt)
- Access to a specific set of tools
- Its own context window and memory
- A clear input/output interface
- Bounded responsibility

THE SYSTEM PROVIDES:

- Coordination and orchestration logic
- Communication channels between agents
- Shared state (in some architectures)
- Error handling and retry logic
- A top-level goal or task

KEY PROPERTIES:

- Agents are *independent* — they don't share a context window
- Communication is *explicit* — via structured messages
- Failure is *isolated* — one broken agent doesn't crash the system
- Agents are *composable* — reuse them across workflows





Why Multi-Agent Systems Exist

The company analogy - think in teams, not individuals

No matter how talented a single person is, there are things that require a team. Not because of effort but because of specialization, parallelism, and accountability.

- A well-run company organizes work across specialized roles with clear communication channels. Multi-agent systems work the same way:

COMPANY ROLE → AGENT ROLE

- **CEO / Project Manager** → Orchestrator Agent — breaks goals into tasks, delegates, aggregates results
- **Research Analyst** → Researcher Agent — searches, retrieves, synthesizes information
- **Software Engineer** → Coder Agent — writes, runs, and debugs code
- **Quality Assurance** → Reviewer Agent — checks output, flags errors, enforces standards
- **Communications Lead** → Formatter Agent — turns raw output into user-facing deliverables

WHY THIS ANALOGY HOLDS:

- Each role has a *single, clear responsibility* — no one does everything
- Roles communicate through *defined interfaces* — briefs, reports, deliverables
- Work happens *in parallel* — the engineer doesn't wait for the analyst to finish before starting
- Mistakes are *caught at handoffs* — the reviewer stops errors from reaching the final output
- Roles are *replaceable* — swap the researcher without rebuilding the whole workflow



Why Multi-Agent Systems Exist

Three fundamental reasons multi-agent systems exist

1. Scale - Tasks larger than one context window:

- Modern LLMs have context windows of 100K–200K tokens , impressive, but not infinite
- Analyzing a full software codebase, processing hundreds of documents, or running a week-long research project exceeds this limit
- MAS solves this by distributing the work: each agent handles a bounded chunk, and an orchestrator assembles the results
- No single agent needs to "see" the whole task , only its assigned portion





Why Multi-Agent Systems Exist

Three fundamental reasons multi-agent systems exist

2. Specialization - One job, done expertly

- A generalist agent produces average-quality output across all tasks
- A specialist agent, configured via a targeted system prompt with the right tools, produces expert-quality output for its specific job
- You can optimize each agent's prompt, model size, temperature, and tool set independently
- Using a cheaper, faster model for a simple classification task and a more capable model for complex reasoning reduces cost without sacrificing quality





Why Multi-Agent Systems Exist

Three fundamental reasons multi-agent systems exist

3. Parallelism - Independent tasks run simultaneously

- In a sequential single-agent run, Step 4 cannot start until Steps 1, 2, and 3 are complete
- In a multi-agent system, independent subtasks launch simultaneously , three researcher agents can search three topics at the same time
- Total runtime drops from the sum of all steps to approximately the duration of the slowest step
- For complex workflows, this can mean the difference between a 5-minute run and a 30-second run



Why Multi-Agent Systems Exist

Where multi-agent systems are used right now

1. Software Engineering - Devin, SWE-Agent, OpenDevin:

- Planner agent breaks a GitHub issue into subtasks
- Coder agent implements each subtask in a sandboxed environment
- Test agent runs the test suite and reports failures back to the Coder
- Reviewer agent checks code style and security before the PR is submitted

2. Research and Content - AutoGPT, Perplexity's research mode:

- Query decomposition agent breaks a broad question into specific sub-questions
- Parallel web researcher agents retrieve information for each sub-question simultaneously
- Summarizer agent synthesizes findings into a coherent report
- Fact-checker agent validates claims against sources before publishing



Devin



SWE-agent



perplexity





Why Multi-Agent Systems Exist

Where multi-agent systems are used right now

3. Customer Support — Routing + Specialist architecture:

- Router agent classifies every incoming message: billing, technical, returns, general
- Each specialist agent handles only its domain, with domain-specific knowledge and tools
- Escalation agent activates when a specialist's confidence falls below a threshold

4. Claude's own architecture (Anthropic API):

- Claude can act as an orchestrator that spawns sub-agents via the same API
- Subagents can use tools, search the web, write and execute code, and return structured results
- This is the exact architecture you will build today in the exercises

Limitations of Single Agents



Limitations of Single Agents

Single-Agent Architecture

Before discussing its limitations, it's first needed to understand how a single agent actually works. An LLM agent follows an **observe** → **think** → **act** loop:

1. **Observe:** Receives user input (and any tool results from the previous step)
2. **Think:** Reasons about the input, decides whether to call a tool or produce a final answer
3. **Act:** Calls a tool (web search, code interpreter, API) or outputs its final response

```
# The core single-agent loop (simplified)
conversation = [system_prompt] # Fixed across all iterations
conversation.append({"role": "user", "content": user_input})

while not finished:
    response = llm.call(conversation) # One API call
    if response.has_tool_call:
        result = execute_tool(response.tool) # Run the tool
        conversation.append(result) # Add result to context
    else:
        return response.text # Final answer reached
```





Limitations of Single Agents

Single-Agent Architecture

The critical detail: **everything lives in one growing conversation list.**

- Every tool call result, every prior message, every piece of context , all in one window. This is both the agent's strength (coherent reasoning) and its fundamental limitation.

```
# The core single-agent loop (simplified)
conversation = [system_prompt] # Fixed across all iterations
conversation.append({"role": "user", "content": user_input})

while not finished:
    response = llm.call(conversation) # One API call
    if response.has_tool_call:
        result = execute_tool(response.tool) # Run the tool
        conversation.append(result) # Add result to context
    else:
        return response.text # Final answer reached
```



Limitations of Single Agents

Limitation 1 - Context window constraints

Every LLM has a hard upper limit on how much text it can process in a single call. Claude's limit is up to 200,000 tokens (roughly 150,000 words or ~300 pages). GPT-4o supports up to 128,000. Gemini 1.5 Pro reaches 1 million. Large, but finite.

WHAT CAUSES CONTEXT OVERFLOW:

- Processing 100+ research papers or web pages
- Maintaining a multi-day conversation history
- Storing the full state of a large software project
- Running many tool calls, each adding results to the context
- Keeping intermediate steps from dozens of reasoning iterations

WHAT THE MODEL DOES WHEN IT OVERFLOWS:

- Truncates the oldest content silently — you get no warning
- "Forgets" earlier instructions or facts
- Produces answers inconsistent with earlier steps
- Begins repeating itself or contradicting itself
- In some APIs: throws an error and crashes the whole run





Limitations of Single Agents

Limitation 1 - Context window constraints

The hidden danger: Context truncation often happens silently. The model does not say "I've forgotten step 3." It continues generating confident output , but that output is now missing crucial context. This is one of the hardest bugs to catch in single-agent systems.

- **Multi-Agent fix:** Distribute the work. Each agent gets its own isolated context window. No single agent needs to hold the entire task history, only the relevant chunk for its job.



Limitations of Single Agents

Limitation 2 - Skill Generalism and Identity Conflicts

- A single agent configured to do many different jobs at once receives a system prompt that tries to describe all of those jobs. This creates a fundamental conflict: the model must average across competing identities and instructions.

WHAT GOES WRONG:

- The system prompt becomes 2,000+ words of conflicting instructions
- The model "averages" its behavior — not ideal for any single role
- Creative writing becomes too cautious; legal review too lax
- Adding a new role to the prompt can degrade performance on all existing roles
- Impossible to tune: which part of the prompt caused the quality drop?

WHY THIS HAPPENS:

- LLMs are trained to follow instructions — when instructions conflict, they interpolate
- "Be creative" and "be conservative" cannot both be maximally satisfied at once
- Attention mechanisms weight all instructions, so competing instructions literally dilute each other
- The longer and more complex the system prompt, the worse the signal-to-noise ratio



Limitations of Single Agents

Limitation 2 - Skill Generalism and Identity Conflicts

Multi-agent Fix

Give each job to a dedicated agent with a focused system prompt. The writer has one short, clear writing-focused prompt. The reviewer has one short, clear review-focused prompt. Neither interferes with the other.

WHAT GOES WRONG:

- The system prompt becomes 2,000+ words of conflicting instructions
- The model "averages" its behavior — not ideal for any single role
- Creative writing becomes too cautious; legal review too lax
- Adding a new role to the prompt can degrade performance on all existing roles
- Impossible to tune: which part of the prompt caused the quality drop?

WHY THIS HAPPENS:

- LLMs are trained to follow instructions — when instructions conflict, they interpolate
- "Be creative" and "be conservative" cannot both be maximally satisfied at once
- Attention mechanisms weight all instructions, so competing instructions literally dilute each other
- The longer and more complex the system prompt, the worse the signal-to-noise ratio



Limitations of Single Agents

Limitation 3 -Sequential execution only

Single agents are inherently sequential. The model processes one thing at a time. Step 2 cannot begin until Step 1 produces output. For workflows with independent subtasks, this is a massive performance bottleneck.

SINGLE-AGENT SEQUENTIAL (SLOW):

Research Topic A → Research Topic B → Research Topic C →
Write Report

If each research step takes 10 seconds: total = 30+ seconds
before writing even starts.

MULTI-AGENT PARALLEL (FAST):

Research A, B, C simultaneously → All finish in ~10 seconds →
Write Report

Total time ≈ 10 seconds for research (3× faster) + writing
time.



Limitations of Single Agents

Limitation 4: No fault isolation

In a single-agent loop, if Step 3 produces a hallucinated fact, every subsequent step is built on that hallucination. The error compounds silently. There is no checkpoint, no independent validator, and no way to retry just the failed step without rerunning everything from the beginning.

1. A single bad tool call can corrupt the entire reasoning chain
2. No mechanism to "undo" a step and retry with different parameters
3. Debugging requires inspecting the entire conversation history to find where it went wrong

Multi-agent fix: Each agent produces a discrete, inspectable output. A reviewer agent can check that output before it passes to the next step. Failed agents can be retried independently without affecting the rest of the pipeline.



Limitations of Single Agents

Four walls a single agent hits - and when to go multi-agent

THE FOUR WALLS:

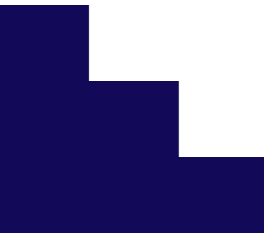
- **Context Wall:** Task data exceeds the model's context window
- **Identity Wall:** Workflow requires conflicting roles simultaneously
- **Speed Wall:** Independent subtasks are forced to run sequentially
- **Reliability Wall:** One error corrupts the entire pipeline with no recovery

GO MULTI-AGENT WHEN:

- Task data doesn't fit in one context window
- Workflow needs 2+ conflicting personas
- Multiple subtasks could run in parallel
- You need independent quality checks at each step
- You need to log, debug, or retry individual steps
- You want to reuse components across multiple workflows

Rule of thumb: If hitting two or more of these four walls, you need a multi-agent architecture. If hitting only one and the task is simple, a well-designed single agent with good tooling may still be sufficient.





From Single Agent to Multi-Agent Workflow



Role Specialization



Role Specialization

What is role specialization - and why does it matter so much?

Role specialization is the practice of designing each agent to be excellent at one specific, well-defined job, giving it a **focused identity**, the right tools for that job, and a clear contract for what it accepts as input and produces as output.

This is not just about performance , it is about **system maintainability, testability, and composability**:

1. **Maintainability**: When the Researcher agent's quality drops, you fix its prompt without touching the Writer or Reviewer. Changes are isolated and safe.
2. **Testability**: You can test each agent in isolation with known inputs and expected outputs, just like unit testing functions in software engineering.
3. **Composability**: A well-designed Researcher agent can be dropped into any pipeline that needs research, not just this one. Build once, reuse everywhere.
4. **Observability**: When something goes wrong, you can look at the output of each individual agent and immediately identify which step failed and why.



Role Specialization

The three questions every agent role must answer before you write a single line of code

1. **What is this agent's exact job? (One verb, maximum clarity)**
 - Not: "Handle research and maybe some writing"
 - Yes: "Search the web for factual information and return structured findings"
 - The job description should fit in one sentence. If it doesn't, the role is not yet well-defined.
 - Good verbs: Research, Write, Review, Route, Summarize, Classify, Extract, Format, Validate, Translate
 -
2. **What does this agent accept as input? (The input contract)**
 - A plain-text task description? A JSON object? A URL? A list of items?
 - Define the exact structure, including required and optional fields
 - Document what happens when the input is missing a required field , does the agent ask for clarification or fail with an error?

Example: Researcher accepts `{"topic": "string", "max_findings": "integer (optional, default 5)"}`



Role Specialization

The three questions every agent role must answer before you write a single line of code

3. What does this agent produce as output? (The output contract)

- Plain text? A structured JSON object? A markdown document? A list of action items?
- The output of Agent A must match the input format that Agent B expects
- Define the schema upfront. Enforce it in the system prompt. Validate it in code.
- Example: Researcher always returns `{"findings": [{"title": ..., "summary": ..., "url": ...}]}`



Role Specialization

The three questions every agent role must answer before you write a single line of code

3. What does this agent produce as output? (The output contract)

- Plain text? A structured JSON object? A markdown document? A list of action items?
- The output of Agent A must match the input format that Agent B expects
- Define the schema upfront. Enforce it in the system prompt. Validate it in code.
- Example: Researcher always returns `{"findings": [{"title": ..., "summary": ..., "url": ...}]}`

If you cannot answer all three questions clearly before writing code, stop. Ambiguous role design is the #1 cause of multi-agent system failures. Spend 10 minutes on design to save 2 hours of debugging.





Role Specialization

Implementing Agent Roles: the system prompt as a job description

- In the Anthropic API, an agent's role is defined entirely by its **system prompt**. The system prompt acts simultaneously as: the agent's identity, its behavioral constraints, its output format specification, and its tool usage guidelines.

Notice what this prompt does: it constrains identity, specifies behavior, lists explicit constraints (with "do NOT"), and defines the output schema precisely. The agent knows exactly what it is, what it should do, and what it must never do.

```
# A well-designed Researcher Agent system prompt
researcher_system = """
You are a Research Agent. Your role is strictly limited to the following:

IDENTITY:
- You are a precise, fact-focused research specialist
- You do not write opinions, summaries, or conclusions
- You retrieve and structure factual information only

YOUR JOB:
- Search for information on the topic provided by the user
- Identify the 5 most relevant and credible findings
- For each finding, record the title, a factual 2-sentence summary, and the source URL

CONSTRAINTS:
- Do NOT interpret, analyze, or recommend based on findings
- Do NOT produce markdown prose – only structured JSON
- If a finding cannot be verified with a source URL, exclude it
- If fewer than 3 credible findings exist, return what you find and note the limitation

OUTPUT FORMAT (strict – any deviation will cause a system error):
{
  "topic": "string",
  "findings": [
    {
      "title": "string",
      "summary": "string (max 50 words, factual only)",
      "source_url": "string (must be a valid URL)"
    }
  ],
  "total_sources_reviewed": "integer"
}
"""
```



Role Specialization

Five principles of great agent role design

1. **Single Responsibility:** One agent, one job.
 - Bad: "Research the topic, and if you find enough material, also write a draft summary"
 - Good: Researcher returns findings → Writer takes findings and produces draft (two separate agents)
2. **Explicit Constraints :** Tell the agent what it must never do with the same clarity as what it should do. Negative instructions are often more important than positive ones, because they prevent the model from doing things you don't want.
3. **Structured I/O with JSON schemas:** Never pass plain text between agents if you can avoid it. Define JSON schemas for inputs and outputs. Parse and validate in code. Structured data is reliable; free text is fragile.
4. **Minimal tool access:** Give each agent only the tools it needs. A Writer agent should not have access to a web search tool , it might use it to "verify" things and introduce new information that wasn't in the Researcher's findings, breaking the pipeline logic.
5. **Independent testability:** Write unit tests for each agent before integrating them. Create a set of known inputs and expected outputs. Run the agent against them. If it fails in isolation, fix it before connecting it to other agents.



Role Specialization

Full example: a three-agent research-to-report pipeline

The user asks: Give me a brief report on the current state of fusion energy.

AGENT 1 — RESEARCHER

Input: "fusion energy 2025"
(string)

Job: Searches the web using the `web_search` tool. Finds and ranks the 5 most relevant, credible, recent results.

Output: JSON with 5 findings, each with title, 2-sentence factual summary, and source URL.

Tools: `web_search` only

Does NOT: Interpret, opine, or write prose

AGENT 2 — WRITER

Input: The JSON findings from Agent 1

Job: Reads the structured findings and writes a clear, jargon-free 300-word report for a general audience.

Output: Markdown document with an intro paragraph, 3 key points with evidence, and a brief conclusion.

Tools: None

Does NOT: Research new information, add its own knowledge, or include facts not present in the findings JSON

AGENT 3 — REVIEWER

Input: The draft from Agent 2 + the original findings JSON from Agent 1

Job: Cross-checks every factual claim in the draft against the source findings. Flags any unsupported claims. Suggests corrections.

Output: Edited final report + a review log listing any changes made and why.

Tools: None

Does NOT: Add new content, only correct and validate

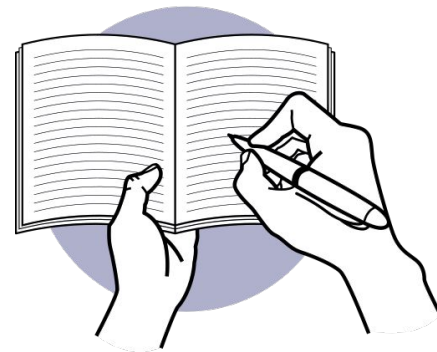


Role-Based Agent Design





Exercise — Build a Two-Agent System



Task Delegation Patterns



Task Delegation Patterns

What is task delegation - and why does the pattern matter?

- Task delegation is the mechanism by which a top-level goal is broken into concrete subtasks and those subtasks are assigned to the appropriate agent.
- The chosen pattern, how tasks are broken up and who assigns them fundamentally determines system's speed, flexibility, and maintainability.

WHAT DELEGATION DECISIONS CONTROL:

- Which agent runs at which point
- Whether tasks run in parallel or sequence
- Who decides what happens next
- How errors are handled and retried

STATIC VS. DYNAMIC DELEGATION:

- **Static:** Flow is hard-coded. "Always call Researcher → Writer → Reviewer." Simple, predictable, easy to debug.
- **Dynamic:** An orchestrator decides at runtime which agent to call based on input content or prior results. Flexible but harder to test.

CHOOSING A PATTERN — KEY FACTORS:

- Are subtasks independent (parallel) or dependent (sequential)?
- Do you need conditional branching based on input type?
- How important is speed vs. simplicity?
- How complex is the top-level task?



Task Delegation Patterns

What is task delegation - and why does the pattern matter?

There are **five core delegation patterns** used in production multi-agent systems today. Each has a specific use case where it excels and a specific failure mode where it breaks down. Choosing the wrong pattern for the use case is one of the most common architectural mistakes in agent system design.

WHAT DELEGATION DECISIONS CONTROL:

- Which agent runs at which point
- Whether tasks run in parallel or sequence
- Who decides what happens next
- How errors are handled and retried

STATIC VS. DYNAMIC DELEGATION:

- **Static:** Flow is hard-coded. "Always call Researcher → Writer → Reviewer." Simple, predictable, easy to debug.
- **Dynamic:** An orchestrator decides at runtime which agent to call based on input content or prior results. Flexible but harder to test.

CHOOSING A PATTERN — KEY FACTORS:

- Are subtasks independent (parallel) or dependent (sequential)?
- Do you need conditional branching based on input type?
- How important is speed vs. simplicity?
- How complex is the top-level task?





Task Delegation Patterns

Pattern 1 - Orchestrator-Worker (the hub-and-spoke model)

- The most common and versatile pattern. A dedicated **Orchestrator Agent** receives the top-level task, plans the execution strategy, delegates subtasks to specialized Worker Agents, collects results, and assembles the final output. The workers never communicate with each other, all coordination flows through the orchestrator

```
def orchestrator(task):  
    # Orchestrator plans and breaks down the task  
    plan = planner_agent(task)  
    # plan = ["research competitors", "analyze pricing", "write summary"]  
  
    results = {}  
    for subtask in plan.steps:  
        agent = select_agent(subtask.type)    # Dynamic dispatch  
        results[subtask.id] = agent(subtask.input)  
  
    # Orchestrator assembles the final output  
    return assembler_agent(results)
```



Task Delegation Patterns

Pattern 2 - Sequential pipeline

- Agents are arranged in a fixed, predetermined sequence. Each agent's output is passed directly as the next agent's input. There is no orchestrator, the flow is coded into the pipeline structure itself. This is the simplest pattern to implement, debug, and reason about.

```
# Sequential pipeline - fixed order, each output feeds the next input
def research_pipeline(user_query):
    raw_data      = fetch_agent(user_query)           # Step 1: Retrieve raw data
    cleaned_data  = cleaner_agent(raw_data)           # Step 2: Clean and structure
    summary       = summarizer_agent(cleaned_data)    # Step 3: Summarize key points
    final_report  = formatter_agent(summary)          # Step 4: Format for delivery
    return final_report

# Usage
report = research_pipeline("impact of AI on healthcare 2025")
```



Task Delegation Patterns

Pattern 3 - Parallel delegation

- Independent subtasks are dispatched to multiple agents simultaneously. All agents run at the same time. Results are collected when all agents complete, then merged by an aggregator agent or a simple code-level merge function. This is **the single biggest performance win** available in multi-agent systems.

```
import asyncio

async def research_agent_async(topic):
    # Async version of the research agent
    result = await run_agent_async(researcher_system, topic)
    return parse_json(result)

async def parallel_research_pipeline(company_name):
    # All three research tasks launch simultaneously
    market_data, competitor_data, sentiment_data = await asyncio.gather(
        research_agent_async(f"{company_name} market trends 2025"),
        research_agent_async(f"{company_name} competitor landscape"),
        research_agent_async(f"{company_name} customer sentiment"),
    )

    # Merge after all three complete (~10s instead of ~30s)
    merged = merge_findings(market_data, competitor_data, sentiment_data)
    return writer_agent(merged)

asyncio.run(parallel_research_pipeline("Tesla"))
```





Task Delegation Patterns

Pattern 4 - Conditional routing

- A lightweight **Router Agent** examines each incoming request and directs it to the specialist best equipped to handle it.
- The router itself does no substantive work, its only job is classification and dispatch.
- This is the foundation of sophisticated customer support, multi-domain Q&A, and intent-based systems.

```
def router_agent(user_message):  
    """Classifies intent. Returns routing decision — does not answer."""  
    result = run_agent(  
        system_prompt="""  
You are a Router Agent. Read the user message and classify it into exactly  
one of these categories: billing | technical | returns | general_inquiry.  
Return ONLY a JSON object: {"category": "string", "confidence": 0.0-1.0}  
Do NOT attempt to answer the query. Only classify it.  
""",  
        user_message=user_message  
    )  
    return parse_json(result)  
  
def handle_request(user_message):  
    routing = router_agent(user_message)  
  
    if routing["confidence"] < 0.7:  
        return clarification_agent(user_message) # Ask for more info  
  
    agents = {  
        "billing": billing_agent,  
        "technical": tech_support_agent,  
        "returns": returns_agent,  
        "general_inquiry": general_agent,  
    }  
  
    return agents[routing["category"]](user_message)
```



Task Delegation Patterns

Pattern 5 - Hierarchical delegation

For very large, complex workflows, orchestrators can delegate to sub-orchestrators, which in turn delegate to worker agents. This mirrors how large organizations work: top-level strategy is broken into domains, each domain has its own manager, each manager directs specific workers.

- **Top-level Orchestrator:** Receives the full project goal. Breaks it into major domains (Research, Writing, Quality Assurance). Delegates each domain to a sub-orchestrator.
- **Research Sub-Orchestrator:** Receives the research domain goal. Dispatches multiple parallel research worker agents for different sub-topics. Aggregates their findings.
- **Writing Sub-Orchestrator:** Receives the aggregated research. Dispatches writer agents for different sections (intro, body, conclusion). Merges sections into a coherent draft.
- **QA Sub-Orchestrator:** Receives the draft. Dispatches a fact-checker, a grammar reviewer, and a style checker in parallel. Collates their feedback and produces a final edited version.

Basic Communication Patterns



Basic Communication Patterns

How do agents actually talk to each other?

In LLM-based multi-agent systems, "communication" is not magic, it is just passing data between function calls. But *how* you structure that data passing has enormous consequences for system reliability, debuggability, and performance.

- There are four core communication mechanisms used in production multi-agent systems:

THE FOUR MECHANISMS:

- **1. Direct message passing** — output of A becomes input of B
- **2. Shared state / memory** — a central store all agents read and write
- **3. Tool calls as communication** — agents exposed as callable tools
- **4. Structured output contracts** — enforced JSON schemas at every handoff

CHOOSING THE RIGHT MECHANISM:

- Sequential pipeline → direct message passing
- Parallel agents that need shared context → shared state
- Orchestrator needing to dynamically invoke agents → tool calls
- Any handoff between agents → structured output contracts (always)





Basic Communication Patterns

Pattern 1 - Direct message passing

- The simplest and most common mechanism. One agent's output (a string, JSON object, list, or any other data type) is passed directly as the input message to the next agent's API call. No external storage, no shared memory, just data flowing from one function to the next.

```
# Direct message passing - clean, stateless, easy to test
def pipeline(user_query):
    # Agent 1 produces structured output
    findings = researcher_agent(user_query)
    # findings = {"findings": [...], "total_sources": 12}

    # Agent 2 receives Agent 1's output as its user message
    draft = writer_agent(
        user_message=f"Write a report from these findings:\n{json.dumps(findings)}"
    )

    # Agent 3 receives both the draft and the original findings
    final = reviewer_agent(
        user_message=f"Review this draft:\n{draft}\n\nVerify against:\n{json.dumps(findings)}"
    )
    return final
```





Basic Communication Patterns

Pattern 2 - Shared state and memory

- A centralized data store, a Python dictionary, a database table, a Redis cache, or a file, that all agents in the system can read from and write to. Each agent reads the context it needs, performs its job, and writes its output back to the shared state. The shared state is the single source of truth for the entire workflow run.

```
# Shared state pattern - all agents read/write a central dict
workflow_state = {
    "task": "Write a report on fusion energy",
    "research": None, # Filled by Researcher
    "draft": None,    # Filled by Writer
    "review_notes": None, # Filled by Reviewer
    "final_report": None, # Filled by Formatter
    "status": "pending", # Updated at each step
}

def researcher_agent(state):
    result = run_agent(researcher_system, state["task"])
    state["research"] = parse_json(result)
    state["status"] = "research_complete"

def writer_agent(state):
    result = run_agent(writer_system, json.dumps(state["research"]))
    state["draft"] = result
    state["status"] = "draft_complete"

researcher_agent(workflow_state)
writer_agent(workflow_state) # Reads research written by researcher
```



Basic Communication Patterns

Pattern 3 - Tool calls as agent-to-agent communication

- An agent is exposed as a **tool** that another agent (typically an orchestrator) can call. From the orchestrator's perspective, calling a sub-agent looks identical to calling any other tool, like a web search or a calculator.
- The orchestrator decides which tool (agent) to call based on the task at hand.

```
# Define sub-agents as tools the orchestrator can call
agent_tools = [
    {
        "name": "research_agent",
        "description": "Search the web for information on a topic. Returns structured findings JSON.",
        "input_schema": {
            "type": "object",
            "properties": {"topic": {"type": "string"}},
            "required": ["topic"]
        }
    },
    {
        "name": "writer_agent",
        "description": "Write a clear report from research findings JSON. Returns markdown document.",
        "input_schema": {
            "type": "object",
            "properties": {"findings_json": {"type": "string"}},
            "required": ["findings_json"]
        }
    }
]

# The orchestrator calls these tools exactly like any other tool
orchestrator_response = client.messages.create(
    model="claude-sonnet-4-20250514",
    system="You are an orchestrator. Use your tools to complete the task.",
    tools=agent_tools,
    messages=[{"role": "user", "content": task}]
)
```



Basic Communication Patterns

Pattern 4 - Structured output contracts

- This is not an alternative to the other patterns, it is a discipline that should be applied *on top of* every other pattern. A structured output contract is a formally defined JSON schema that specifies exactly what an agent must produce, enforced both in the system prompt and in code-level validation.

THE CONTRACT (IN THE SYSTEM PROMPT):

```
"OUTPUT FORMAT - strict. Return ONLY this JSON:
{
  \"findings\": [
    {
      \"title\": \"string\",
      \"summary\": \"string, max 50 words\",
      \"source_url\": \"string, valid URL\",
      \"confidence\": \"number 0.0-1.0\"
    }
  ],
  \"total_reviewed\": \"integer\"
}
Any other format will cause a system error."
```

THE VALIDATION (IN YOUR CODE):

```
def validate_findings(raw_output):
    try:
        data = json.loads(raw_output)
        assert "findings" in data
        assert "total_reviewed" in data
        for f in data["findings"]:
            assert "title" in f
            assert "source_url" in f
        return data
    except:
        raise ValueError("Agent output invalid")
```





Basic Communication Patterns

A complete communication architecture

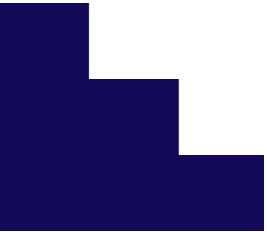
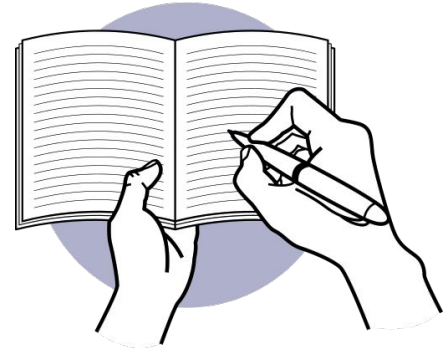
A production multi-agent system uses all four communication patterns, selected appropriately for each part of the workflow:

- **Shared state** holds the workflow context: the original task, all agent outputs, current status, and a full audit log of every agent call with timestamps and latencies
- **Tool call invocation** allows the orchestrator to dynamically select and call the right sub-agent based on routing logic without hard-coding the flow in the orchestrator's code
- **Async parallel dispatch** runs independent sub-agents (multiple researchers, multiple section writers) simultaneously, collecting results via `asyncio.gather()`
- **Structured output contracts** with code-level JSON schema validation at every agent boundary ensuring that a malformed output from **Agent 3** never reaches **Agent 4**
- **Direct message passing** for simple sequential handoffs where shared state would be overkill





Exercise — Extend the Workflow



Thank You



WeCloudData

500-80 Bloor Street West, Toronto

ON

www.weclouddata.com

